



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS

LICENCIATURA EN INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Asignatura:
Sistemas Operativos

Asignación:
Laboratorio #2: Gestión de Procesos, Señales y la Interfaz /proc

Profesor:
Prof. Isacio Tamayo

Estudiante:
James Tack

Grupo:
1IL143

Fecha de entrega:
11 de septiembre de 2026

2do Semestre 2026

De qué se trata el laboratorio

En este laboratorio la máquina Linux deja de ser algo que solo se instala y pasa a ser el objeto de estudio. El trabajo tiene tres bloques. Primero se genera carga con procesos que consumen CPU y se leen sus PID, PPID y estados, siguiendo la jerarquía hasta el PID 1. Después se prueban las señales POSIX contra un script que las intercepta con `trap`. En el tercero se abre `/proc/<PID>/` para revisar descriptores, cambios de contexto, límites y el tamaño "fantasma" de sus pseudo-archivos. Por el camino hay dos retos de depuración, uno con un pipeline y otro con permisos, y al final un cuestionario de nueve preguntas.

El entorno: por qué no usé la VM del laboratorio 1

Mi plan era seguir en la máquina virtual `vm-sistemas-operativos` que dejé armada con Vagrant en el laboratorio 1, pero no arrancó. Primero se reanudó desde el estado guardado y el SSH nunca respondió. La apagué, la encendí en frío y esta vez el kernel se quedó detenido en `Begin: Loading essential drivers ...`. El registro de VirtualBox explica el motivo en una sola línea.

```
HM: HMR3Init: Attempting fall back to NEM: AMD-V is not available
```

En el anfitrión está activo Hyper-V, que lo usan WSL2 y Docker Desktop. Mientras Hyper-V controla el procesador, VirtualBox no puede usar AMD-V directamente y cae al modo NEM, que pasa todo por la API de Hyper-V y resulta tan lento que el arranque no termina nunca. Arreglarlo exigía apagar Hyper-V con `bcdedit` y reiniciar el equipo, lo que a su vez deja sin funcionar a WSL y a Docker. Opté por hacer el laboratorio en Ubuntu sobre WSL2, que ya tenía instalado. Para lo que pide la guía no hay diferencia, porque es un kernel Linux real con `/proc`, señales POSIX, `systemd` como PID 1 y las mismas herramientas.

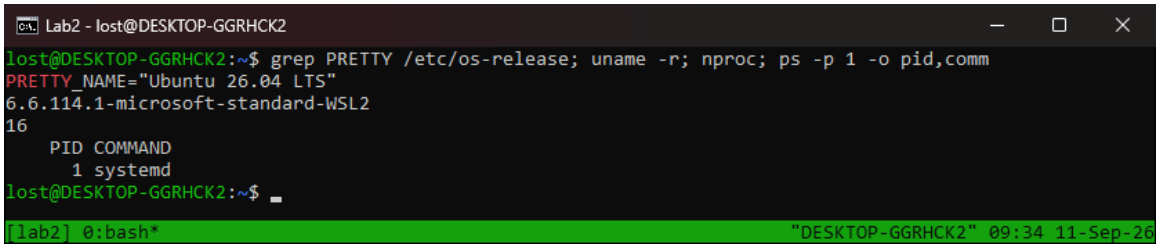


Figura 1. Entorno de trabajo: Ubuntu 26.04 LTS sobre WSL2, kernel 6.6.114.1, 16 CPU y systemd como PID 1.

Ejecuté todo dentro de una sola sesión de `tmux` llamada `lab2`, que es la barra verde que aparece abajo en cada captura junto con la hora. Usar siempre el mismo shell tiene una razón práctica: su PID, el 456, aparece como padre en varias fases.

Preparación

La guía pide actualizar los repositorios e instalar los tres paquetes que traen las herramientas del laboratorio.

```
sudo apt update && sudo apt install -y procps psmisc coreutils
```

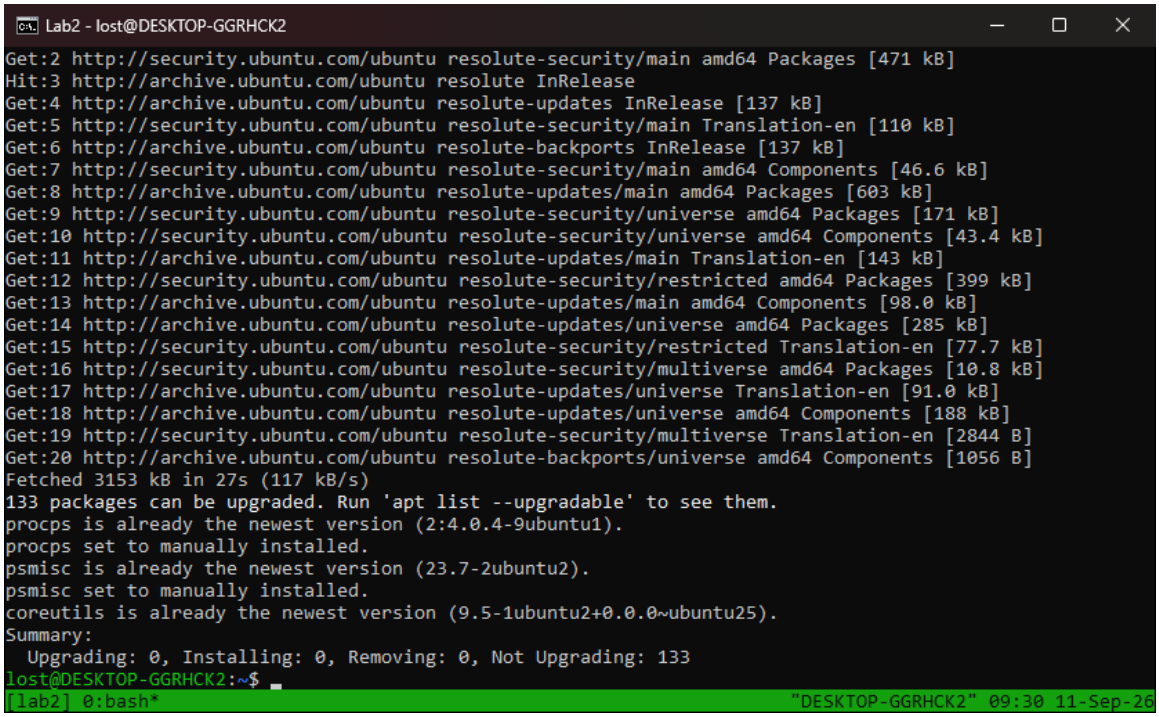


Figura 2. Actualización de repositorios; procps, psmisc y coreutils ya estaban en su última versión.

Los tres venían con la imagen de Ubuntu, y `apt` se limitó a marcarlos como instalados manualmente (`Installing: 0`). De `procps` salen `ps`, `pgrep` y `pkill`, de `psmisc` sale `pstree`, y `coreutils` aporta `sha256sum` y `sleep`. Más adelante instalé también `strace` para una comprobación extra de la fase 3.

Fase 1. Carga, pipelines, jerarquía y estados

3.1 Generación y verificación de carga sintética

```
for i in {1..4}; do (sha256sum /dev/urandom > /dev/null &); done
ps aux | grep sha256sum
```

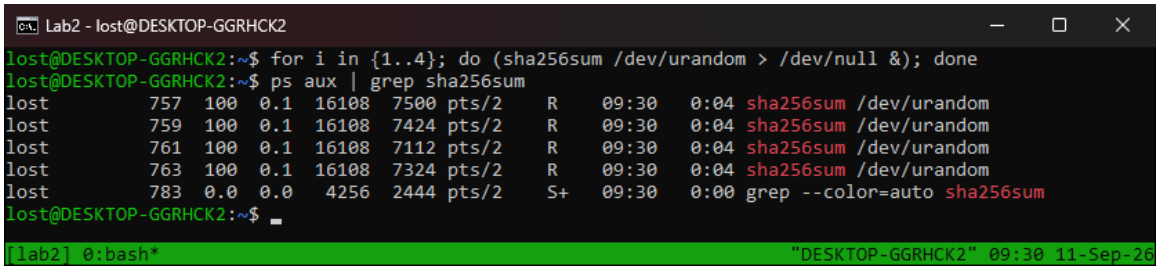


Figura 3. Los cuatro procesos sha256sum en ejecución y la línea del propio grep.

En cada vuelta del bucle los paréntesis abren un subshell, y dentro de él se lanza `sha256sum` en segundo plano. Como lee de `/dev/urandom`, que nunca se acaba, y su salida termina en `/dev/null`, el proceso no hace otra cosa que calcular hashes. Es carga de CPU pura.

Proceso	PID	%CPU	STAT
sha256sum /dev/urandom	757	100	R

Proceso	PID	%CPU	STAT
sha256sum /dev/urandom	759	100	R
sha256sum /dev/urandom	761	100	R
sha256sum /dev/urandom	763	100	R

Los cuatro marcan 100 %. En `ps` el `%CPU` es el tiempo de CPU consumido dividido entre el tiempo que lleva vivo el proceso, medido por núcleo. Esta máquina tiene 16 CPU y el planificador pudo darle un núcleo entero a cada `sha256sum` sin que tuvieran que compartir. En un equipo de dos núcleos se repartirían la CPU entre los cuatro y cada uno rondaría el 50 %.

Por qué aparece `grep sha256sum`. En un pipeline el shell crea todos los procesos a la vez y no uno detrás del otro. Cuando `ps` recorre `/proc` para armar la lista, `grep` ya existe, y como su línea de comandos contiene el texto `sha256sum`, `ps` lo incluye y `grep` encuentra la coincidencia en su propia línea. De ahí sale el PID 783 con estado `S+`, dormido mientras espera que le llegue texto por la tubería y dentro del grupo de primer plano. Se evita con `pgrep sha256sum` o con el truco `grep [s]ha256sum`, cuyo patrón ya no coincide con el texto literal de su propio comando.

3.2 Reto de depuración #1: corrección del pipeline

```
ps aux --sort=-%cpu | awk '{print $2, $3, $11}' | head -n 3
```

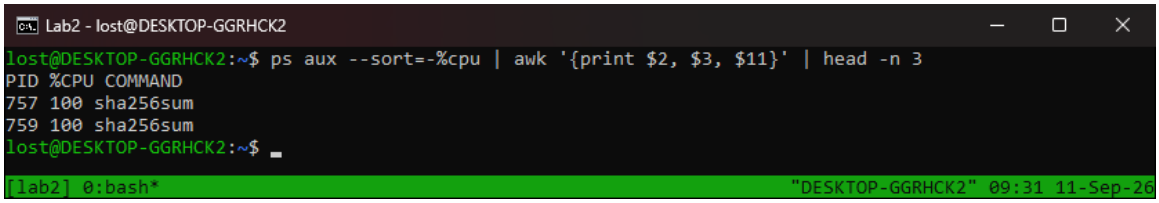


Figura 4. Pipeline original: la cabecera ocupa una de las tres líneas que deja pasar head.

Se piden tres procesos y salen dos. El fallo está en cómo cuenta cada etapa del pipeline.

- `ps aux --sort=-%cpu` imprime primero una línea de cabecera (`USER PID %CPU ...`) y después los procesos, de mayor a menor consumo.
- `awk` no distingue entre cabecera y datos y aplica `{print $2, $3, $11}` a todas las líneas. En la cabecera, los campos 2, 3 y 11 son justamente los títulos `PID`, `%CPU` y `COMMAND`.
- `head -n 3` cuenta líneas y no procesos. La primera de las tres es la cabecera, de modo que en pantalla quedan solo dos procesos reales (757 y 759).

Para corregirlo hay que quitar la cabecera antes de que llegue a `head`. Probé dos formas de hacerlo.

```
ps aux --sort=-%cpu | awk 'NR>1 {print $2, $3, $11}' | head -n 3
ps -eo pid,pcpu,comm --sort=-pcpu --no-headers | head -n 3
```

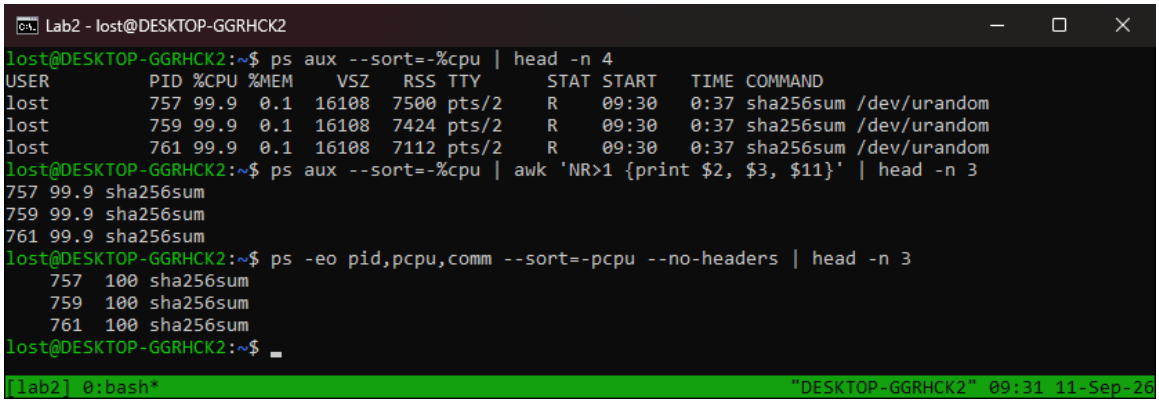


Figura 5. Salida cruda de ps, pipeline corregido con awk y alternativa en la que ps no imprime cabecera.

En la primera, `NR` es el número de línea que va contando `awk`. Con la condición `NR>1` se salta la línea 1, que es la cabecera, y de las demás imprime solo PID, %CPU y comando. Etapa por etapa queda así:

- `ps aux --sort=-%cpu` lista todos los procesos del sistema ordenados por %CPU descendente; el signo menos es lo que invierte el orden.
- `awk 'NR>1 {print $2, $3, $11}'` descarta la cabecera y se queda con las columnas 2, 3 y 11, que son el PID, el %CPU y el nombre del ejecutable.
- `head -n 3` deja pasar las tres primeras líneas, que ahora sí son tres procesos. Al llegar a la tercera cierra la tubería, y los procesos anteriores terminan con `SIGPIPE` en vez de seguir trabajando para nada.

La segunda forma me parece más limpia porque deja que `ps` haga el trabajo. Con `-o pid,pcpu,comm` elige las columnas, `--sort=-pcpu` ordena y `--no-headers` suprime la cabecera, con lo que `awk` sobra. Además, `comm` es el nombre del proceso y no el campo 11 de una línea partida por espacios, que se descuadra si alguna columna anterior cambia de formato. Las dos variantes devuelven 757, 759 y 761 como los tres procesos con más CPU.

3.3 Análisis de estados del kernel

```
ps -eo pid,ppid,stat,comm
```

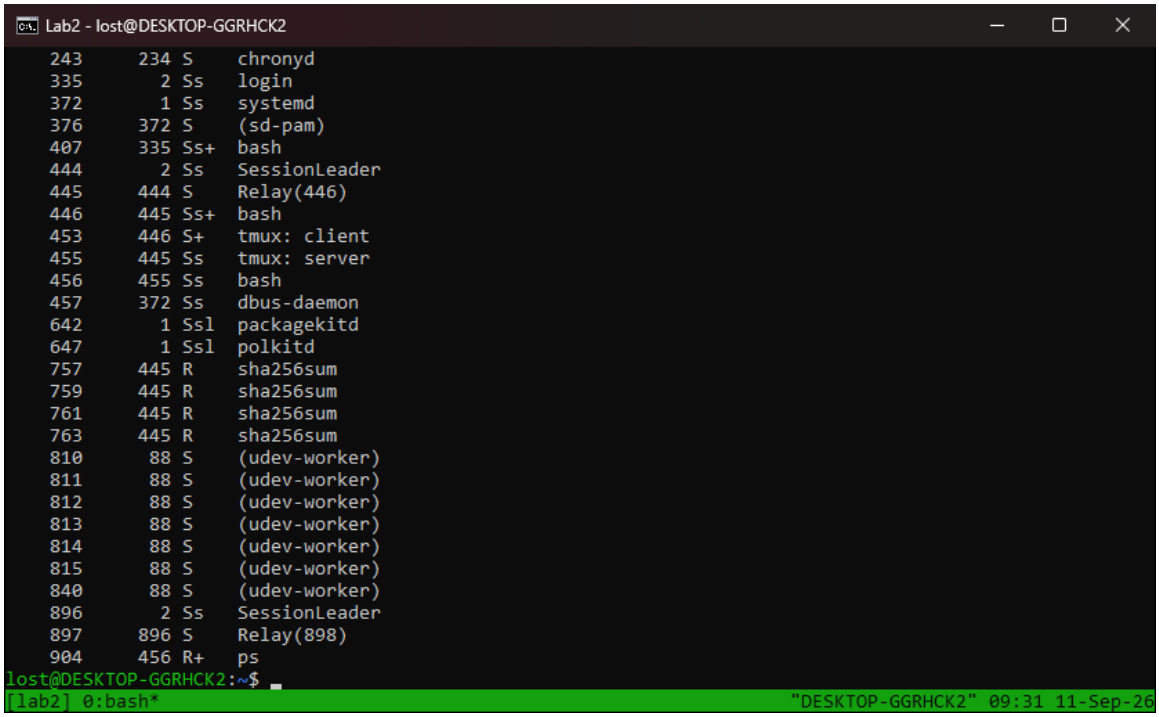


Figura 6. Parte final de ps -eo pid,ppid,stat,comm.

De la lista escogí tres procesos con estados distintos.

PID	PPID	STAT	Nombre	Lectura
757	445	R	sha256sum	En ejecución o en la cola de listos. Nunca espera nada: siempre tiene datos que procesar.
456	455	Ss	bash	Dormido interrumpible esperando lo que yo escriba, y líder de sesión (s).
647	1	Ssl	polkitd	Demonio dormido esperando peticiones, líder de sesión (s) y con varios hilos (l).

Los caracteres extra los busqué en la sección PROCESS STATE CODES de `man ps`.

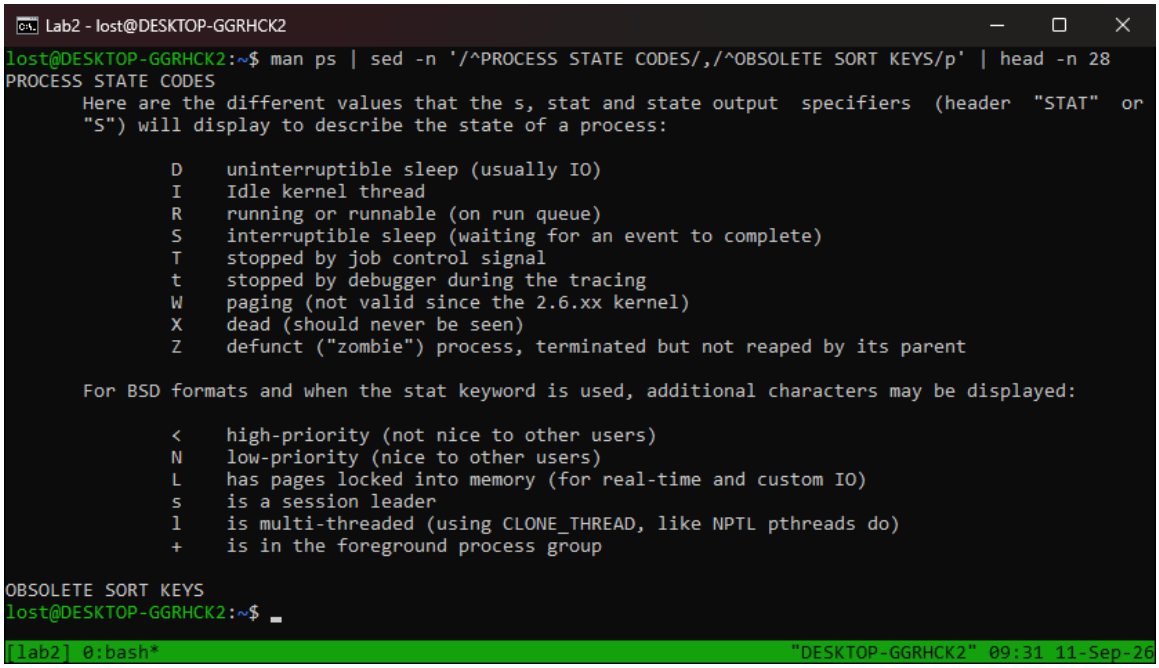


Figura 7. Sección PROCESS STATE CODES de man ps.

- `s` indica que el proceso es líder de sesión, es decir, que su PID coincide con el ID de sesión. Suelen serlo los shells de login y los demonios que llaman a `setsid()`, como el `bash` 456 y el `systemd` 372.
- `+` marca que pertenece al grupo de procesos de primer plano de su terminal, el que recibe lo que se teclea y las señales de `Ctrl+C`. En la captura lo tienen `tmux: client` (`S+`) y el propio `ps` (`R+`), mientras que los `sha256sum` no, porque están en segundo plano.
- `l` significa que el proceso tiene varios hilos creados con `CLONE_THREAD`, como `packagekitd` y `polkitd` (`Ssl`).
- `<` señala prioridad alta (un valor nice negativo) y `N`, lo contrario. No apareció ninguno de los dos en mi lista porque en WSL no hay demonios con el nice modificado.

Además, el propio `ps` (904) aparece con `R+`. Mientras lee `/proc` para armar la lista está usando la CPU, y por eso se ve a sí mismo corriendo.

3.4 Inspección de la jerarquía de procesos

```
echo "PID de mi shell: $$"
pgrep -n sha256sum
pstree -p -s $(pgrep -n sha256sum)
```

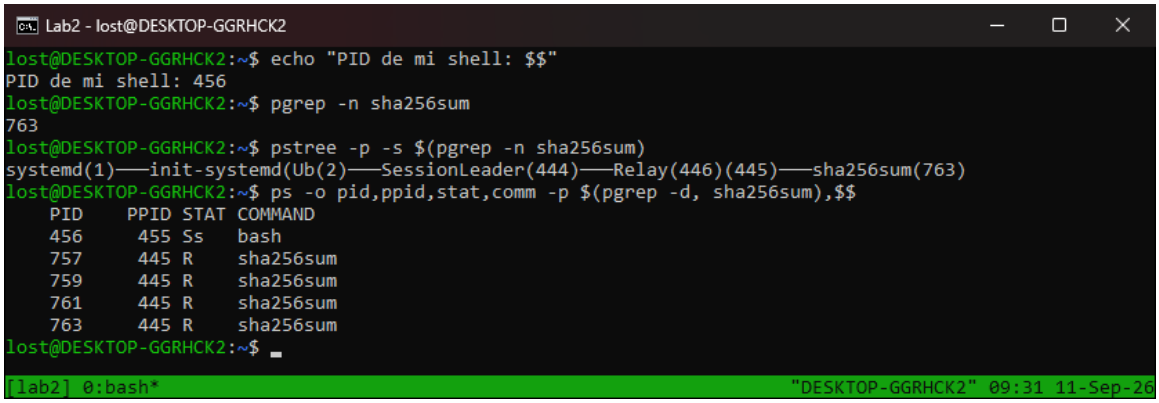


Figura 8. PID del shell, último sha256sum creado y su árbol hasta el PID 1.

`pgrep -n` devuelve el más nuevo de los procesos que coinciden, que es el 763, y mi intérprete de comandos es el `bash` con PID 456. Sin embargo, el árbol de `sha256sum` no pasa por mi shell, cosa que no esperaba.

```
systemd(1)---init-systemd(Ub(2)---SessionLeader(444)---Relay(446)(445)---sha256sum(763)
```

El padre de los cuatro `sha256sum` es el PID 445 y no el 456. En `pstree`, `Relay(446)` es el nombre del proceso y el PID es el número entre paréntesis que viene después. La causa está en los paréntesis del bucle de la fase 3.1: `( ... & )` crea un subshell hijo de `bash`, ese subshell lanza `sha256sum` en segundo plano y termina de inmediato. `sha256sum` se queda huérfano, y el kernel se lo asigna al ancestro más cercano marcado como *subreaper* con `prctl(PR_SET_CHILD_SUBREAPER)`, o al PID 1 si no hay ninguno. En WSL ese papel lo cumple el proceso `Relay` 445 de la sesión, mientras que en un Ubuntu normal lo adoptaría `systemd --user` o el propio PID 1. El servidor de `tmux` (455) cuelga del mismo 445 por la misma razón, ya que se desprende de la terminal con un doble `fork`.

Para ver una relación padre-hijo directa lancé un proceso sin subshell.

```
sleep 300 &
pstree -p -s -l $!
```

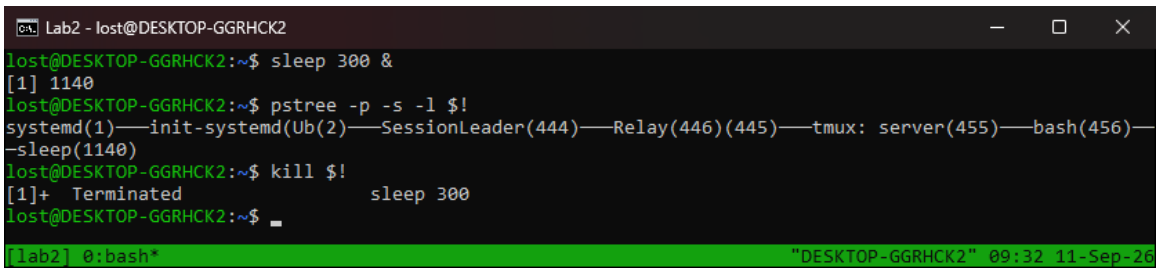


Figura 9. Un sleep lanzado directamente sí cuelga del bash 456.

Esta vez sí aparece `bash(456)---sleep(1140)`. El shell hace `fork()`, el hijo reemplaza su imagen con `exec("sleep")` y queda con PPID 456. En los dos casos la cadena termina en `systemd(1)`, la raíz de todo el espacio de usuario. Todo proceso tiene un padre, y si ese padre muere antes, lo adopta alguien más arriba en la cadena, en último caso el PID 1. El `init-systemd(Ubuntu)` con PID 2 es el init propio de WSL, que prepara el entorno y le deja el PID 1 a `systemd`.

3.5 Limpieza de carga mediante tuberías

```
pgrep sha256sum | xargs kill -s SIGTERM
ps aux | grep sha256sum
```

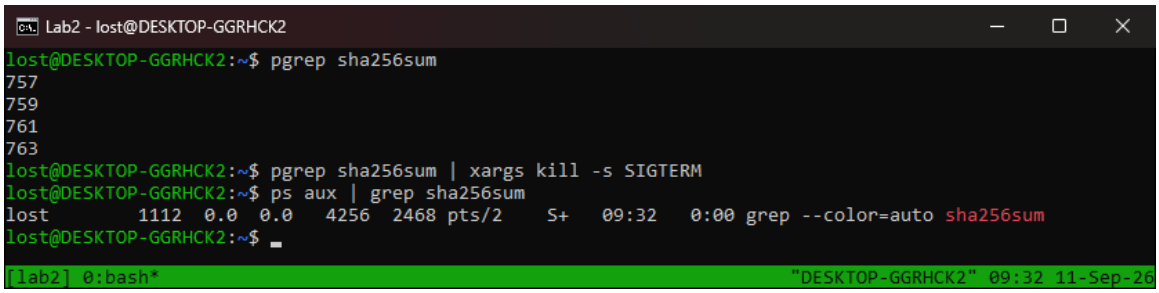


Figura 10. Limpieza con pgrep | xargs kill y verificación: solo queda la línea de grep.

- **Qué produce `pgrep`.** Los PID de los procesos cuyo nombre coincide con el patrón, uno por línea (757, 759, 761 y 763), sin nada más, justamente para poder encadenarlo con otro comando.
- **Qué hace `xargs`.** Lee palabras de su entrada estándar y las añade como argumentos a un comando. Aquí arma y ejecuta `kill -s SIGTERM 757 759 761 763` en una sola llamada.
- **La relación entre los dos.** `kill` no lee su entrada estándar, solo sus argumentos, y con `pgrep sha256sum | kill` los PID se perderían. `xargs` hace de puente y convierte la salida de `pgrep` en la lista de argumentos de `kill`.
- **Qué significa `SIGTERM`.** Es la señal 15, la petición educada de terminar. Por defecto finaliza el proceso, aunque este puede capturarla para cerrar archivos y limpiar antes de salir, o incluso ignorarla. `sha256sum` no instala manejador y muere en el acto.

Después de la limpieza, `ps aux | grep sha256sum` solo muestra el `grep`. Tampoco salió ningún aviso `Terminated` del shell, porque esos procesos ya no eran trabajos (*jobs*) de este `bash`: desde el principio los había adoptado el 445.

Fase 2. Señales POSIX y captura con trap

4.1 Creación del script de pruebas

Creé `servicio_senales.sh` con `nano` en mi carpeta personal.

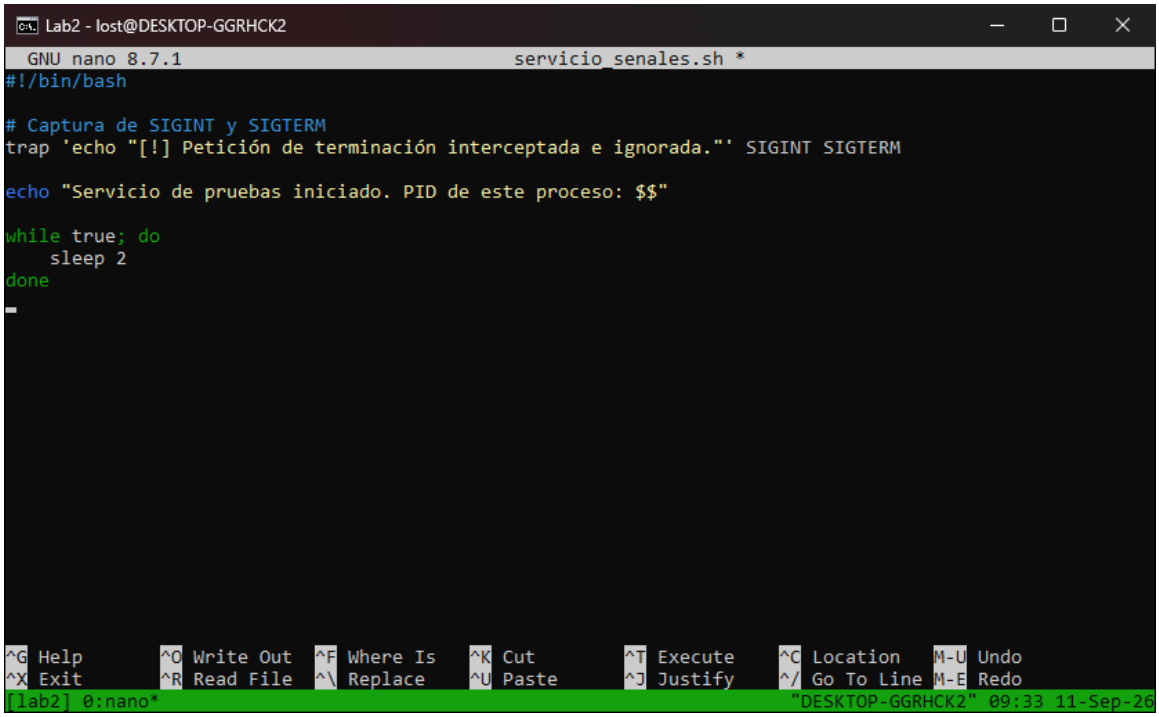


Figura 11. servicio_senales.sh en nano, antes de guardar.

La línea importante es la del `trap`. Le indica a `bash` que, si le llegan `SIGINT` o `SIGTERM`, ejecute el `echo` en lugar de morir. Luego el script imprime su PID con `$$` y entra en un bucle infinito que duerme de dos en dos segundos.

4.2 Reto de depuración #2: error de permisos

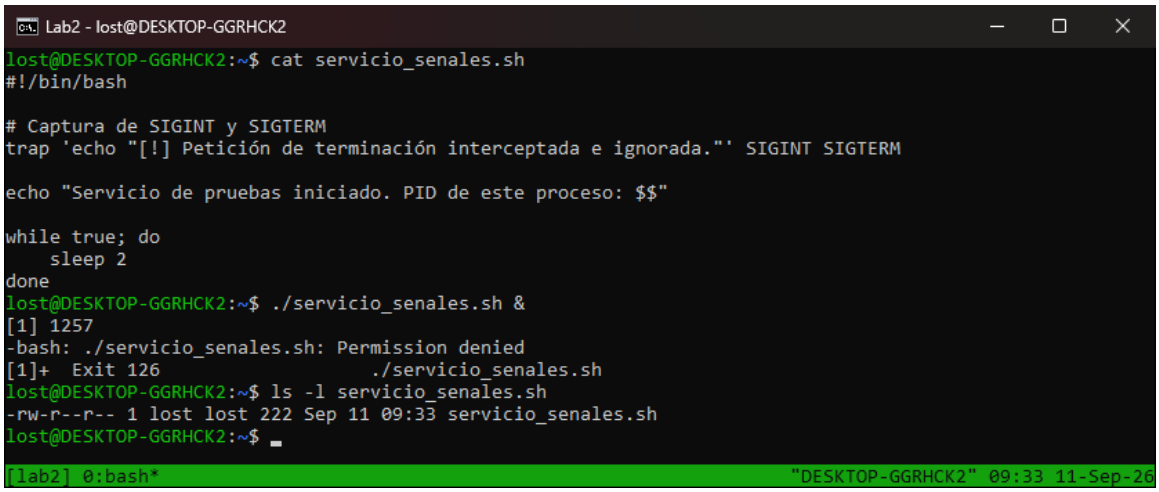


Figura 12. Primer intento: Permission denied, código de salida 126 y permisos -rw-r--r--.

El intento falla con `Permission denied` y el shell informa `Exit 126`, que en `bash` quiere decir que encontró el archivo pero no lo puede ejecutar. La razón se ve en `ls -l`.

```
-rw-r--r-- 1 lost lost 222 Sep 11 09:33 servicio_senales.sh
```

- El primer carácter, `-`, dice que es un archivo regular.

- `rw-` son los permisos del propietario (`lost`), que puede leer y escribir pero no ejecutar.
- `r--` son los del grupo `lost`, solo lectura.
- `r--` son los del resto de usuarios, también solo lectura.

No hay ninguna `x`. Al escribir `./servicio_senales.sh` el shell llama a `execve()`, y el kernel revisa el bit de ejecución antes de cargar nada; como falta, devuelve `EACCES`. Los archivos nuevos nunca nacen con ese bit, porque `nano` los crea con modo 666 y el `umask` 022 lo recorta a 644. Da igual que el contenido sea un script correcto.

La solución es cambiar el modo del archivo y no su dueño, que sería trabajo de `chown`. Usé `chmod` en forma simbólica para añadir solo el permiso de ejecución al propietario.

```
chmod u+x servicio_senales.sh
```

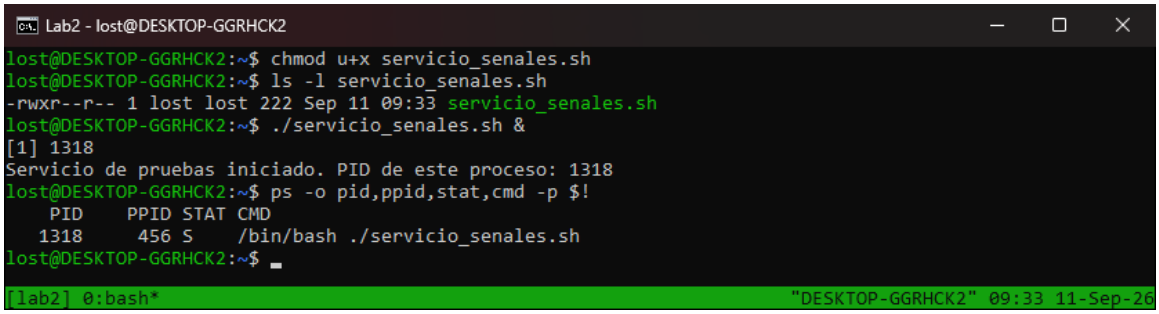


Figura 13. Permisos -rwxr--r-- después de chmod u+x y ejecución del script con PID 1318.

El archivo pasa a `-rwxr--r--`, 744 en octal. La `x` aparece solo en el primer grupo y el propietario sigue siendo `lost`. `chmod +x` también habría funcionado, pero le daría ejecución a todo el mundo, y para esta prueba basta con que la tenga yo. El script arrancó e imprimió `PID de este proceso: 1318`, con PPID 456 (mi shell) y estado `S`, dormido en su `sleep 2`. Otra salida habría sido `bash servicio_senales.sh`, donde el ejecutable es `bash` y el script solo se lee, con lo que basta el permiso `r`. El reto, en cambio, pide ejecutarlo directamente.

4.3 Evaluación de resistencia a señales

```
kill -2 1318
kill -15 1318
ps aux | grep servicio_senales
```

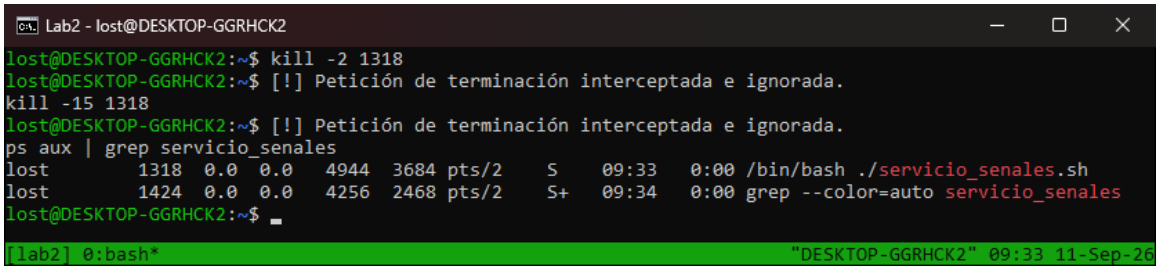


Figura 14. SIGINT y SIGTERM interceptados por trap; el proceso 1318 sigue vivo.

Con cada señal el script respondió `[!] Petición de terminación interceptada e ignorada.` y siguió corriendo, y `ps` lo muestra todavía con PID 1318 y estado `S`. El mensaje no sale al instante sino después del prompt, con el siguiente comando ya a medio escribir. Esto pasa porque `bash`

recibe la señal mientras espera a que termine su hijo `sleep 2`, anota que llegó y ejecuta el manejador del `trap` cuando ese `sleep` vuelve. El retraso puede llegar a dos segundos, que es lo que se ve en la captura.

- **Qué hace `trap`.** Registra un manejador para una o varias señales. Por debajo `bash` usa `sigaction()`, y cuando la señal llega el kernel le devuelve el control al shell para que ejecute el texto indicado en lugar de aplicar la acción por defecto.
- **Qué habría pasado sin él.** La acción por defecto de `SIGINT` (2) y de `SIGTERM` (15) es `Term`, terminar el proceso, según la tabla de `man 7 signal` de la Figura 16. El script habría muerto con la primera señal.
- **Por qué sigue vivo.** El manejador solo hace `echo` y no llama a `exit`, de modo que al terminar el bucle `while true` continúa donde estaba.

4.4 Destrucción incondicional del proceso

```
kill -l
man 7 signal
```

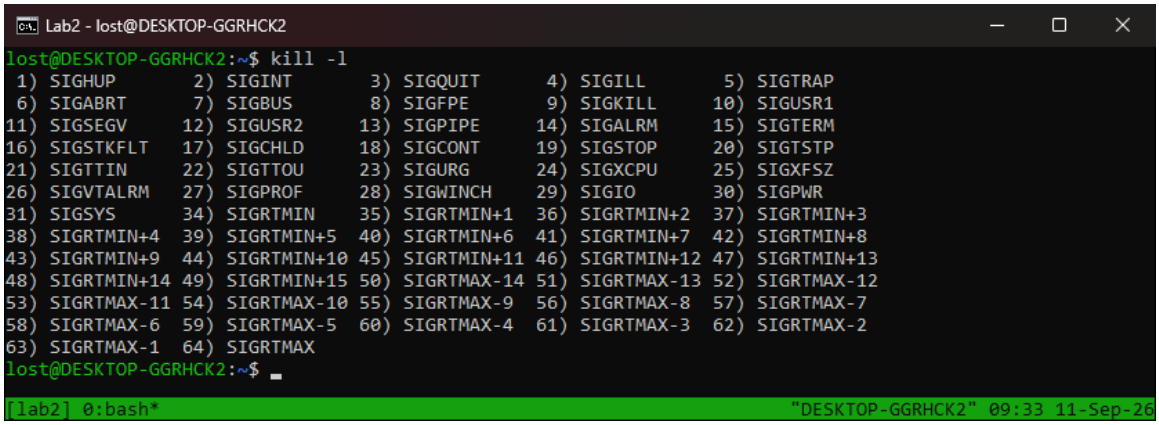


Figura 15. Lista de señales con kill -l.

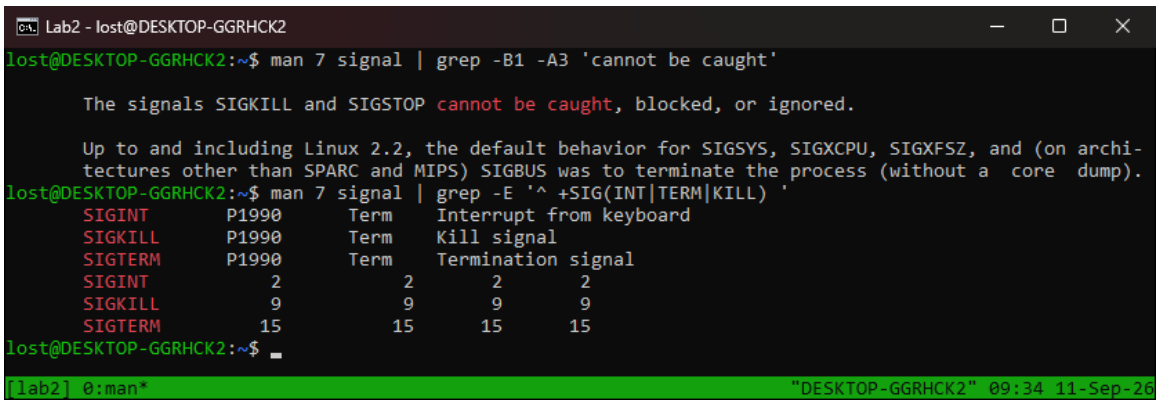


Figura 16. man 7 signal: SIGKILL y SIGSTOP no se pueden capturar, bloquear ni ignorar.

`man 7 signal` lo dice textualmente: *The signals SIGKILL and SIGSTOP cannot be caught, blocked, or ignored*. De esas dos, `SIGSTOP` solo detiene el proceso, y la que sirve para terminarlo es `SIGKILL`.

Dato	Valor
Nombre	SIGKILL
Número	9
Comando usado	`kill -9 1318`
Acción	Term, ejecutada por el kernel sin pasar por el proceso

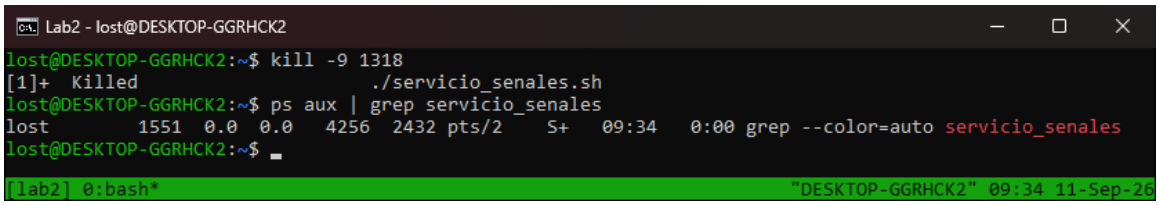


Figura 17. kill -9 termina el script; en ps ya solo queda la línea de grep.

El shell avisó `[1]+ Killed ./servicio_senales.sh` y `ps` ya no muestra el 1318. Ese `Killed` lo escribe el `bash` padre al recoger con `waitpid()` el estado de salida del hijo, que indica que murió por la señal 9.

Por qué `trap` no la intercepta. Con las demás señales el kernel consulta la tabla de manejadores del proceso y, si encuentra uno, le pasa el control. Con `SIGKILL` esa consulta no se hace. `sigaction()` rechaza con `EINVAL` cualquier intento de instalarle un manejador o de ignorarla, y tampoco se puede enmascarar con `sigprocmask()`. Cuando llega, el kernel despierta al proceso aunque esté dormido en `sleep` y lo termina él mismo, sin ejecutar una sola instrucción del programa. Gracias a eso el administrador siempre puede detener cualquier proceso. Sin embargo, el precio lo paga el proceso, que no puede borrar sus temporales, cerrar sus conexiones con cuidado ni guardar su estado.

Fase 3. /proc: información del proceso, FD y métricas

5.1 y 5.2 Proceso con redirección de flujos y sus descriptores

```
sleep 10000 > /dev/null 2>&1 &
echo $!
P=$!
cd /proc/$P/
ls -l fd/
```

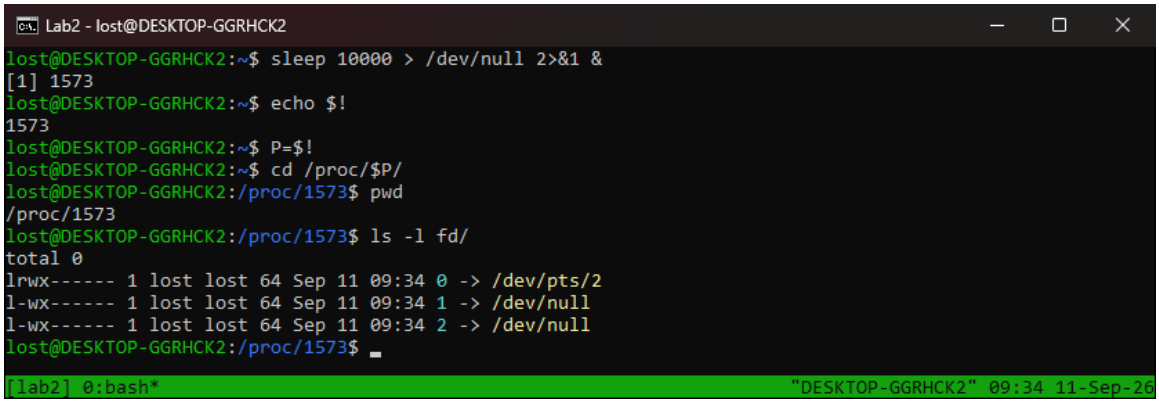


Figura 18. Proceso sleep 1573 y sus descriptores en /proc/1573/fd.

El `sleep` quedó con PID 1573, que guardé en la variable `P` para no teclearlo en el resto de la fase. En `/proc/1573/fd/` hay un enlace simbólico por cada descriptor abierto.

Descriptor	Canal POSIX	Ruta o dispositivo	Modo del enlace
0	stdin	/dev/pts/2	lrwx (lectura y escritura)
1	stdout	/dev/null	l-wx (solo escritura)
2	stderr	/dev/null	l-wx (solo escritura)

Por qué 1 y 2 apuntan a `/dev/null`. Antes del `exec` de `sleep`, el shell procesa las redirecciones de izquierda a derecha. `> /dev/null` abre `/dev/null` para escritura y lo coloca en el descriptor 1 con `dup2()`, y luego `2>&1` copia el descriptor 1 en el 2, con lo que los dos apuntan al mismo archivo abierto. No obstante, el orden importa. Con `2>&1 > /dev/null` el error seguiría saliendo por la terminal, porque el 2 habría copiado al 1 cuando todavía era la pantalla. El 0 no se redirigió y `sleep` heredó la entrada del shell, la pseudoterminal `/dev/pts/2` de mi panel de `tmux`. Los permisos de cada enlace reflejan el modo en que se abrió el archivo, y `/dev/null` se abrió solo para escribir.

5.3 Auditoría de cambios de contexto

```
grep -i "switches" status
```

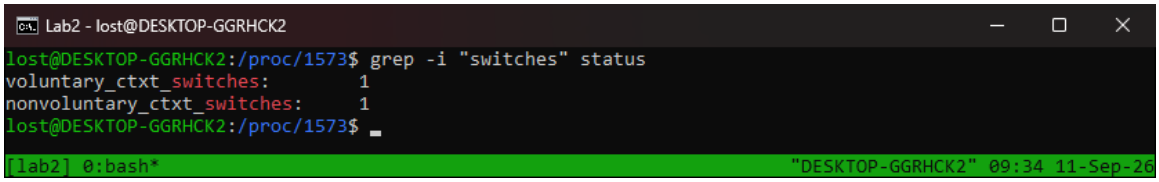


Figura 19. Cambios de contexto del proceso sleep.

Un cambio de contexto es el momento en que el kernel deja de ejecutar un proceso en una CPU y pone otro en su lugar. Para poder retomarlo después guarda sus registros, su contador de programa y su pila, y carga los del siguiente. No hace trabajo útil y cuesta tiempo, pero sin él 16 núcleos no podrían atender a cientos de procesos.

- **Voluntario.** El proceso cede la CPU porque no puede seguir. Pidió algo que todavía no está listo, como leer del disco, esperar una tecla o dormir un rato, y se bloquea.

- **Involuntario.** El proceso podía seguir trabajando pero el planificador le quitó la CPU, porque se le acabó su porción de tiempo o porque despertó otra tarea con más derecho a ese núcleo.

Para mi `sleep` los dos contadores valen 1, ninguno es mayor que el otro. Encaja con lo que hace `sleep`, que es casi nada. Arranca, carga sus bibliotecas, llama a `clock_nanosleep()` pidiendo 10000 segundos y se bloquea, y ese bloqueo es su único cambio voluntario. El involuntario debió de ocurrir al arrancar, en el milisegundo entre el `fork` y el `exec`, cuando el planificador lo desplazó una vez. Mientras duerma, los contadores no se van a mover: un proceso que no corre no cede nada ni puede ser desalojado.

Para tener con qué comparar medí un proceso que hace lo contrario, un `sha256sum` durante cinco segundos.

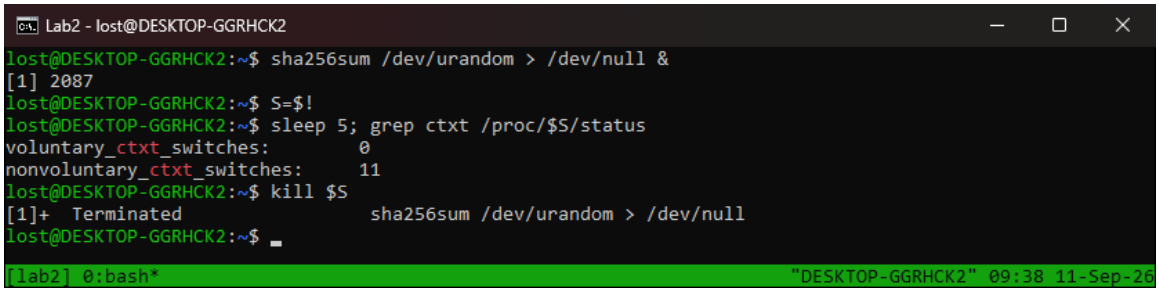


Figura 20. Contraste: un sha256sum tras cinco segundos de ejecución.

Dio cero voluntarios y 11 involuntarios. `sha256sum` nunca espera, porque `/dev/urandom` siempre tiene datos, y por eso jamás suelta la CPU por su cuenta. Cada vez que la pierde es porque el planificador (EEVDF en este kernel 6.6) se la quitó. Sin embargo, son pocos porque, con 16 CPU casi libres, casi nadie le disputaba su núcleo; en un equipo cargado el número se dispararía.

5.4 Comparativa: ps frente a /proc

```
ps -p $P -o pid,ppid,stat,comm
grep -E "^(Name|State|Pid|PPid):" status
```

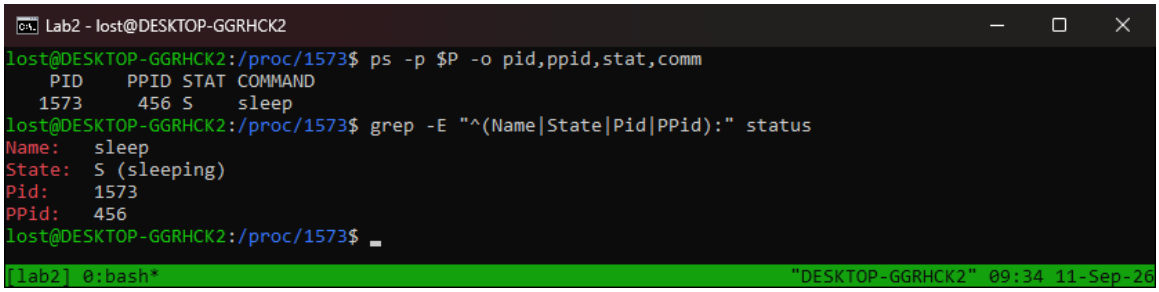


Figura 21. El mismo proceso visto con ps y leyendo /proc/1573/status.

Campo	Valor reportado por ps	Valor en /proc/1573/status
PID	1573	Pid: 1573
PPID	456	PPid: 456
Estado	S	State: S (sleeping)
Nombre	sleep	Name: sleep

Los cuatro valores coinciden porque `ps` no tiene otra fuente. No existe una llamada al sistema que devuelva "la lista de procesos". `ps` recorre los directorios numéricos de `/proc`, lee `stat`, `status` y `cmdline` de cada uno y da formato al texto. Lo comprobé con `strace`, que muestra qué archivos abre `ps`.

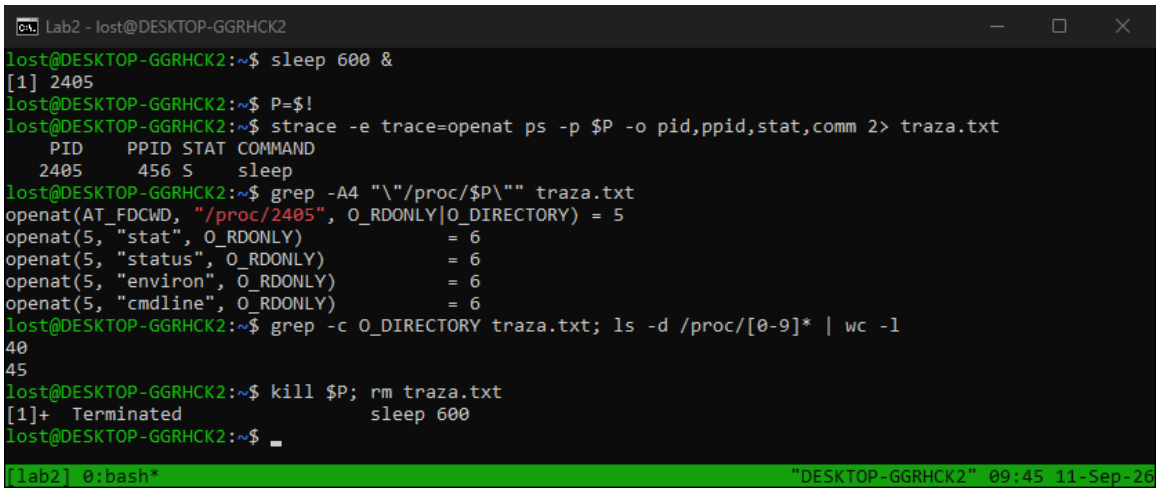


Figura 22. strace sobre ps: los archivos de /proc que abre para mostrar un solo proceso.

Para mostrar el proceso 2405, `ps` abre el directorio `/proc/2405` y, relativos a él, los archivos `stat`, `status`, `environ` y `cmdline`. Dos de ellos, `stat` y `status`, son justo los que yo había leído a mano con `grep`. La opción `-p` no le ahorra trabajo. La traza registra 40 aperturas de directorios `/proc/<n>`, que son prácticamente todos los procesos del sistema (45 cuando los conté un momento después, con algunos procesos más vivos). O sea, `ps` recorre la lista completa, lee cada proceso y descarta los que no coinciden con el PID pedido.

La diferencia está en la presentación. `/proc` entrega los datos crudos en el formato que decidió el kernel, por ejemplo `State: S (sleeping)`, y `ps` los filtra, los alinea en columnas y les añade cálculos propios. El `%CPU` es uno de ellos: no está en `/proc`, sale de dividir tiempos que sí están ahí.

5.5 Evaluación de límites del proceso

```
cat limits | head -n 12
```

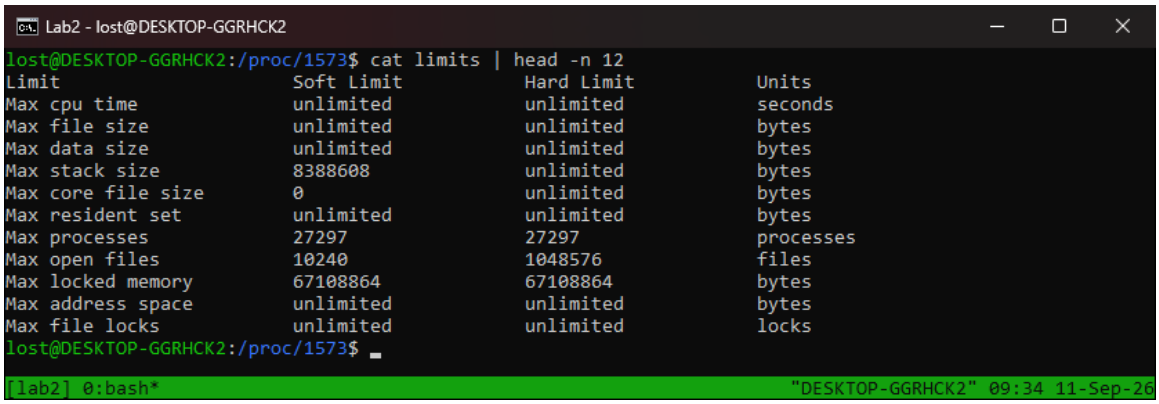


Figura 23. Primeras 12 líneas de /proc/1573/limits.

Límite	Soft Limit	Hard Limit	Unidad
Max open files	10240	1048576	archivos
Max processes	27297	27297	procesos

- **Soft limit.** Es el límite que el kernel aplica en este momento. Si el proceso intenta abrir el archivo 10241, `open()` falla con `EMFILE`. El proceso puede subirlo con `setrlimit()`, o con `ulimit -n` desde el shell, pero nunca por encima del hard.
- **Hard limit.** Es el techo del soft. Un usuario normal puede bajarlo pero no volver a subirlo, porque para eso hacen falta privilegios (`CAP_SYS_RESOURCE`). En archivos abiertos hay mucho margen, de 10240 a 1048576, mientras que en procesos los dos valores son 27297 y ese límite no se estira sin ser root.

Otros valores de la tabla también dicen algo. `Max core file size` tiene soft 0, y si el proceso se cae no deja volcado de memoria en disco. `Max stack size` es de 8 MB, por lo que una recursión infinita acaba en `Segmentation fault` en lugar de devorar la RAM. Todos estos límites se heredan de padre a hijo, y `sleep` tiene los de mi `bash`.

Por qué importan en un sistema multiusuario. El hardware se comparte, y sin límites el primer programa mal escrito o malicioso se lo queda todo. Una *fork bomb* llenaría la tabla de procesos hasta que nadie pudiera abrir ni una terminal, y aquí se estrella contra los 27297 de `Max processes`. Un servidor con una fuga de descriptores terminaría agotando los del sistema, y uno que pide memoria sin parar obliga al *OOM killer* a matar procesos, quizá de otro usuario. Además, nada impide a un proceso descontrolado llenar el disco. Con los límites, esos fallos se quedan como un error dentro del proceso culpable y la máquina sigue en pie.

5.6 Experimento: el tamaño "fantasma" de /proc

```
ls -lh status
cp status ~/status_copia.txt
ls -lh ~/status_copia.txt
wc -c status ~/status_copia.txt
```

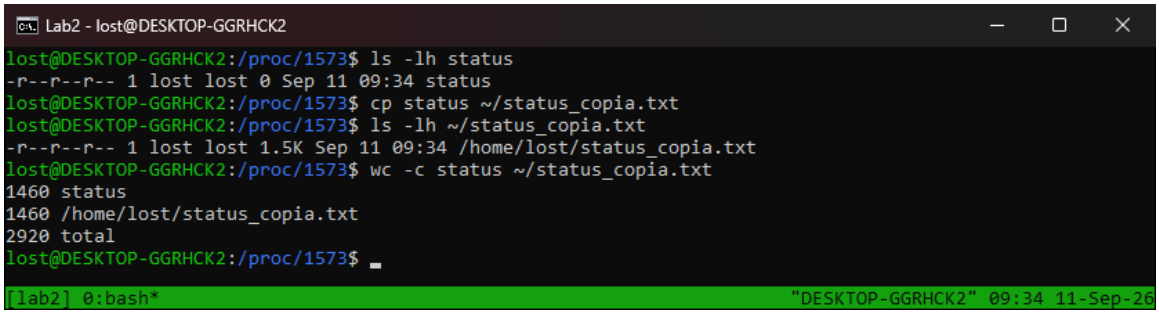


Figura 24. El pseudo-archivo reporta 0 bytes; la copia mide 1.5K y wc lee 1460 bytes de ambos.

- **Tamaño del pseudo-archivo.** `ls -lh` muestra `0` bytes.
- **Tamaño de la copia.** `1.5K`, que según `wc -c` son 1460 bytes exactos.
- **Por qué el pseudo-archivo dice 0.** `ls` no lee el contenido, llama a `stat()` y muestra el campo de tamaño del inodo. Los inodos de `/proc` no tienen datos detrás, porque el texto no existe en

ningún sitio hasta que alguien lo pide y el kernel no sabe cuánto medirá sin generarlo. Por eso reporta 0.

- **Por qué la copia sí tiene tamaño.** `cp` no se fía del tamaño y lee hasta que `read()` devuelve 0, el fin de archivo. Los 1460 bytes que recibió los escribió en un archivo normal de ext4 que ocupa bloques reales en disco y cuyo inodo sí guarda un tamaño. `wc -c` también lee todo el contenido, de ahí que dé 1460 en los dos casos.

Qué pasa en el `read()`. La llamada entra al VFS, que ve que el archivo pertenece a procfs y usa sus operaciones en lugar de las de un sistema de archivos en disco. En el caso de `status`, procfs ejecuta una función del kernel que busca el `task_struct` del proceso 1573 y escribe en un búfer, en ese instante, su nombre, estado, PID, memoria, contadores de cambios de contexto y demás datos. El texto se copia al espacio de usuario y se descarta, y si se vuelve a leer se genera otra vez con los valores del momento. `/proc` es entonces una ventana a las estructuras del kernel y no un archivo, y la copia, en cambio, es una foto fija. Dentro de una hora `status_copia.txt` seguirá diciendo lo mismo aunque el proceso real haya cambiado o ya no exista. La copia, por cierto, heredó el modo `-r--r--r--` del original, porque `cp` conserva los bits de permiso de la fuente.

5.7 Destrucción del proceso y verificación de /proc

```
cd ~
kill -SIGKILL $P
ps -p $P
ls /proc/$P
```

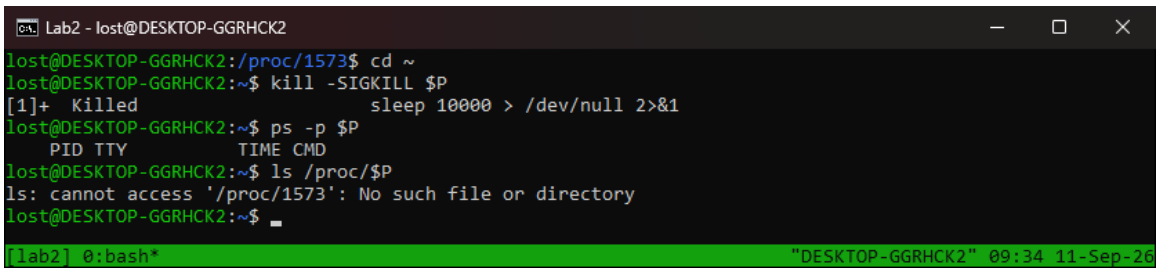


Figura 25. Tras SIGKILL, el proceso y su directorio en /proc desaparecen.

Antes de matarlo salí de `/proc/1573` con `cd ~`, porque si no el shell se habría quedado con su directorio de trabajo apuntando a algo que iba a dejar de existir.

- **El proceso.** El shell avisó `[1]+ Killed sleep 10000 > /dev/null 2>&1`, y `ps -p 1573` solo imprime la cabecera porque ya no hay ningún proceso con ese PID.
- **`/proc/1573`.** `ls` responde `No such file or directory`, el directorio desapareció junto con el proceso.

Otra vez queda claro que `/proc` no se guarda en ningún lado. Cuando se busca `/proc/1573`, procfs revisa en la tabla de procesos del kernel si hay una tarea con ese PID. Si la hay, construye el directorio al vuelo, y si no, responde que no existe. Visto como ciclo de vida, el directorio aparece con el `fork()` y dura mientras el proceso vive. Si el proceso termina y su padre todavía no lo ha recogido, sigue ahí como zombie, con `State: Z`, hasta que el padre llama a `wait()` y el kernel libera el `task_struct`. En este caso el padre era mi `bash` interactivo, que recoge a sus hijos enseguida (de ahí el `Killed`), y ni siquiera llegó a quedar un zombie.

Cuestionario técnico

1. Cambios de contexto

Voluntario. El ejemplo más claro es el `bash` de mi sesión. Cuando muestra el prompt llama a `read()` sobre la terminal y, como no hay nada escrito, el kernel lo marca como dormido (`S`), lo saca de la cola de ejecución y pone otro proceso en la CPU. Le pasa lo mismo al `sleep` de la fase 3, que se bloquea en `clock_nanosleep()`, y a cualquier servidor que espera en `accept()` la llegada de un cliente. El proceso se va porque no tiene nada que hacer hasta que ocurra algo externo.

Involuntario. Un proceso limitado por CPU como `sha256sum` siempre tiene trabajo y nunca se bloquea por sí mismo. Sin embargo, el planificador lo desaloja cuando consume su porción de tiempo o cuando despierta una tarea que debe correr antes en esa CPU, por ejemplo el shell al recibir una tecla. El sistema tiene que repartir el procesador con justicia, y si el planificador no pudiera quitarle la CPU a nadie, un solo bucle infinito congelaría el núcleo. En mis mediciones, `sha256sum` acumuló 11 involuntarios y 0 voluntarios en cinco segundos, frente a 1 y 1 de `sleep` en toda su vida.

2. Naturaleza de /proc

Los pseudo-archivos de `/proc` son interfaces hacia el kernel y no archivos guardados. Un archivo en disco tiene un inodo que registra cuántos bytes ocupa, mientras que el inodo de un pseudo-archivo no tiene datos detrás y su contenido se fabrica en el momento del `read()`, a partir de las estructuras internas del proceso. Como ese texto no existe antes de la lectura y además cambia de una lectura a otra, el kernel no puede anunciar un tamaño y reporta 0. Las herramientas que leen hasta el fin de archivo, como `cat`, `cp` o `wc`, reciben el contenido completo, y las que se fían del campo de tamaño, como `ls -l` o `stat`, ven 0. En mi prueba `status` marcó 0 bytes y entregó 1460.

3. Procesos zombie

Un zombie es un proceso que ya terminó. Llamó a `exit()`, y el kernel le liberó la memoria, cerró sus descriptores y soltó sus recursos, pero conserva en la tabla de procesos una entrada mínima con su PID, su código de salida y sus estadísticas de uso. Esa entrada existe para que el padre pueda preguntar con `wait()` cómo terminó su hijo, y mientras no lo haga `ps` muestra el proceso como `Z` o `<defunct>`.

Lo reproduje con un padre que nunca llama a `wait()`: un `bash` que lanza `sleep 1` en segundo plano y después se reemplaza a sí mismo con `exec sleep 60`.

```
bash -c 'sleep 1 & exec sleep 60' &
```

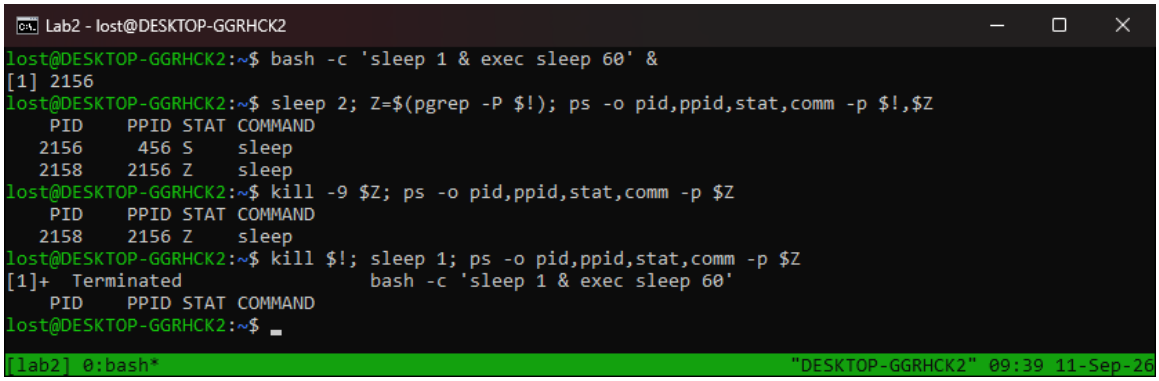


Figura 26. Zombie 2158: sobrevive a kill -9 y desaparece cuando muere su padre 2156.

Al segundo, el hijo 2158 terminó y quedó en `Z`, colgando del 2156. Le envié `kill -9` y siguió en `Z`. `kill -9` no puede eliminarlo porque ya no queda nada que matar. Una señal actúa sobre un programa en ejecución, y el zombie no tiene código, ni memoria, ni hilo que el kernel pueda interrumpir; es solo un registro esperando a que alguien lo lea. Para que desaparezca el padre tiene que llamar a `wait()` o `waitpid()`. Si no lo hace nunca, queda la opción de terminar al padre, y entonces el zombie pasa al PID 1 o a un *subreaper*, que sí recoge a sus hijos y libera la entrada. Es lo que se ve en la última línea de la figura, donde maté al 2156 y el 2158 dejó de existir.

4. Manejo de señales

Sin `trap`, un proceso que recibe `SIGTERM` o `SIGINT` aplica la acción por defecto, que en ambos casos es terminar, y el kernel lo finaliza sin que el programa se entere. Con `trap`, `bash` registra un manejador que cambia esa disposición, y la señal pasa a interrumpir lo que el proceso estaba haciendo para ejecutar el código indicado. En `servicio_senales.sh` ese código es un `echo`, y las dos señales se convirtieron en un mensaje mientras el bucle seguía: el 1318 recibió `kill -2` y `kill -15` y continuó en estado `S`. También vi que el manejador de `bash` no corre al instante si el script está esperando un comando externo, y el mensaje apareció cuando terminó el `sleep 2` en curso.

Además, con `trap` se puede ignorar una señal (`trap "" SIGTERM`) o volver al comportamiento normal (`trap - SIGTERM`). En los scripts reales lo habitual es usarlo para limpiar, borrando temporales o liberando un bloqueo antes de salir con `exit`, más que para ignorar señales.

5. SIGTERM frente a SIGKILL

Aspecto	SIGTERM (15)	SIGKILL (9)
Acción predeterminada	Terminar el proceso (Term)	Terminar el proceso (Term)
¿Se puede capturar?	Sí, con trap o sigaction()	No; sigaction() devuelve EINVAL
¿Se puede ignorar o bloquear?	Sí	No
Participación del proceso	El proceso decide: puede limpiar y salir, o seguir corriendo	Ninguna: el kernel lo termina sin ejecutar su código
Cuándo usarla	Primero, siempre: es la forma de pedir un cierre ordenado	Solo si el proceso no responde a SIGTERM

Por defecto las dos terminan el proceso, y lo que cambia es quién tiene la última palabra. `SIGTERM` es una petición que el proceso atiende a su manera, mientras que `SIGKILL` es una orden que ejecuta el

kernel. Con el mismo script lo comprobé: sobrevivió a `kill -15` y cayó con `kill -9`. Por lo tanto, `systemd`, y también `kill` cuando no recibe argumentos, mandan primero `SIGTERM` y dejan `SIGKILL` para el final, ya que con esta última se pierde cualquier oportunidad de guardar datos o cerrar conexiones.

6. Descriptores de archivos en un análisis forense

`/proc/<PID>/fd/` enseña lo que un proceso tiene abierto en este momento, sin depender de sus registros ni de lo que diga el propio programa. Ante un proceso sospechoso sirve para varias cosas.

- **Recuperar archivos borrados.** El malware suele borrar del disco su ejecutable o su carga útil después de abrirlos. El enlace aparece como `/tmp/.x (deleted)`, pero mientras el descriptor siga abierto el contenido es accesible y se puede rescatar con `cp /proc/<PID>/fd/3 evidencia.bin`. Con `/proc/<PID>/exe` pasa lo mismo si borraron el binario.
- **Descubrir conexiones de red.** Los sockets aparecen como `socket:[48213]`. Con ese número de inodo se busca la línea correspondiente en `/proc/net/tcp`, o se usa `ss -p`, y se obtienen la IP y el puerto remotos. Así se descubre, por ejemplo, un proceso de nombre inocente que mantiene una conexión con un servidor de comando y control.
- **Ver qué espía o escribe.** Un descriptor hacia `/dev/input/event*` delata un keylogger, y uno hacia un archivo oculto en `/dev/shm` o `/tmp` puede ser el lugar donde guarda lo que roba. La salida estándar y la de error redirigidas a `/dev/null`, como en la fase 3, apuntan a un proceso que no quiere dejar rastro en la terminal.
- **Seguir tuberías.** Un `pipe:[inodo]` compartido entre dos procesos revela que se comunican aunque no tengan una relación padre-hijo evidente.

7. ps, top, htop y /proc

Las tres herramientas son lectoras de `/proc` y no tienen ningún acceso privilegiado a la información de los procesos; lo único que hacen es darle formato. `ps` saca una foto. Recorre `/proc/[0-9]*`, lee `stat`, `status` y `cmdline` de cada proceso, imprime y termina, como muestra el `strace` de la Figura 22. `top` hace lo mismo en bucle cada pocos segundos, lee además `/proc/stat`, `/proc/meminfo` y `/proc/loadavg` para la cabecera, y calcula el `%CPU` restando los tiempos de CPU de dos lecturas consecutivas. `htop` sigue la misma idea con una interfaz interactiva, barras por núcleo y una vista de hilos que obtiene de `/proc/<PID>/task/`.

Se trata de una relación de capas. El kernel mantiene el estado real en sus estructuras, procfs lo expone como texto y las herramientas lo ordenan para que lo lea una persona. Si `/proc` no está montado, ninguna de las tres funciona.

8. Ciclo de vida de procesos

Estado	Significado	Ejemplo observado
R	En ejecución o listo en la cola del planificador esperando su turno de CPU.	Los sha256sum 757 a 763, que nunca esperan nada; o ps (904), mientras lee /proc.
S	Dormido interrumpible: espera un evento (datos, tiempo, un hijo) y una señal lo puede despertar.	bash 456 esperando una orden; sleep 1573; el script 1318 dentro de su sleep 2.
Z	Terminado, pero su padre aún no ha recogido	El sleep 2158 de la pregunta 3, hijo de un

Estado	Significado	Ejemplo observado
	el estado de salida con wait().	proceso que se reemplazó con exec y nunca llama a wait().

9. Límites de recursos

`/proc/<PID>/limits` muestra los límites de recursos (*rlimits*) que el kernel aplica a ese proceso en concreto, entre ellos CPU, tamaño de archivos, pila, memoria bloqueada, procesos y archivos abiertos, cada uno con su valor blando y su valor duro. Se consultan y modifican con `ulimit`, `prlimit` o `setrlimit()`, y se heredan de padre a hijo, de modo que un límite fijado en el login alcanza a todo lo que ese usuario lance después. El archivo sirve para auditar. Si un servicio falla con `Too many open files`, aquí se comprueba con qué límite arrancó.

Sin estos mecanismos, cualquier proceso podría agotar un recurso compartido y el daño lo sufrirían todos. Del lado de la estabilidad, una fuga de memoria obliga al OOM killer a matar procesos ajenos, una fuga de descriptores deja al sistema sin poder abrir archivos y un error de recursión se come la RAM en lugar de fallar solo. Del lado de la seguridad, un usuario sin privilegios podría tumbar la máquina a propósito con una *fork bomb* (`:(){ :|:& };:`) que llene la tabla de procesos, o llenar el disco con volcados de memoria; por eso aquí `Max core file size` está en 0. Sería una denegación de servicio que no necesita explotar ninguna vulnerabilidad, basta con que no haya límites.

Lista de comprobación

Elemento	Resultado	Evidencia
PIDs y %CPU de la carga	757, 759, 761, 763 al 100 %	Figura 3
Análisis del proceso grep	Se lista a sí mismo por aparecer en /proc junto a ps	Figura 3
Reto #1: pipeline corregido	awk 'NR>1 ...' y alternativa ps --no-headers	Figuras 4 y 5
Estados de procesos	R, Ss, Ssl, S+, R+ explicados	Figuras 6 y 7
Árbol con pstree	sha256sum adoptado por 445; sleep hijo directo de bash 456	Figuras 8 y 9
Limpieza de procesos	pgrep | xargs kill -s SIGTERM	Figura 10
Reto #2: permisos	-rw-r--r-- → chmod u+x → -rwxr--r--	Figuras 12 y 13
SIGINT y SIGTERM	Interceptadas por trap; el proceso sigue vivo	Figura 14
SIGKILL	Señal 9, kill -9 1318, proceso terminado	Figuras 15 a 17
Descriptores 0, 1 y 2	/dev/pts/2, /dev/null, /dev/null	Figura 18
Cambios de contexto	sleep: 1/1; sha256sum: 0/11	Figuras 19 y 20
ps frente a /proc	Valores idénticos; ps lee /proc	Figuras 21 y 22
Límites	Open files 10240/1048576; processes 27297/27297	Figura 23
Pseudo-archivo y copia	0 bytes frente a 1460 bytes	Figura 24
Eliminación del proceso	ps vacío; /proc/1573 ya no existe	Figura 25
Cuestionario técnico	9 preguntas respondidas; zombie reproducido	Figura 26

Conclusiones

Casi nada de lo que muestran las herramientas está guardado en alguna parte, y eso fue lo que más se repitió en el laboratorio. `ps` lee `/proc`, `/proc` le pregunta al kernel en el momento y el archivo `status` mide 0 bytes aunque tenga 1460 caracteres dentro. El directorio del proceso 1573 dejó de existir en el mismo instante en que murió el proceso, porque nunca había estado en disco. Ahora leo una salida de `ps` como lo que es, una foto de las estructuras del kernel tomada una fracción de segundo antes.

La jerarquía me dio la sorpresa del laboratorio. Esperaba que los `sha256sum` salieran como hijos de mi shell, pero los paréntesis del bucle los dejaron huérfanos y los adoptó el proceso `Relay` de WSL, y tuve que lanzar un `sleep` sin subshell para ver la relación padre-hijo que describe la guía. De ahí saqué que el PPID indica quién es responsable de recoger al proceso cuando termine, que no siempre coincide con quién lo lanzó y puede cambiar en plena vida del proceso. La prueba del zombie cerró el tema. Con `kill -9` no conseguí nada, y bastó matar al padre para que el sistema lo limpiara.

Con las señales entendí la diferencia entre pedir y ordenar. El script ignoró tranquilamente dos peticiones de terminar y solo cayó con la señal que el kernel ejecuta sin preguntar. Me llamó la atención el retraso del `trap`, de hasta dos segundos, porque `bash` espera a que termine el `sleep` antes de atender la señal, y en un servicio real una demora así puede importar.

Queda el cambio de entorno. La VM del laboratorio 1 dejó de arrancar porque Hyper-V, que usan WSL2 y Docker, le quitó a VirtualBox el acceso directo a la virtualización del procesador. Es otro supuesto invisible, parecido al de las Guest Additions del laboratorio anterior: el Vagrantfile no cambió y aun así la máquina ya no sirve en este equipo. Por suerte, este laboratorio solo necesitaba Linux y su `/proc`, y WSL2 los trae completos.

Anexos

Anexo A. Comandos ejecutados

Fase	Comando	Para qué
Setup	sudo apt update && sudo apt install -y procps psmisc coreutils	Instalar las herramientas
3.1	for i in {1..4}; do (sha256sum /dev/urandom > /dev/null &); done	Generar carga de CPU
3.1	ps aux | grep sha256sum	Ver PIDs y %CPU
3.2	ps aux --sort=-%cpu | awk '{print $2, $3, $11}' | head -n 3	Pipeline original (con error)
3.2	ps aux --sort=-%cpu | awk 'NR>1 {print $2, $3, $11}' | head -n 3	Pipeline corregido
3.2	ps -eo pid,pcpu,comm --sort=-pcpu --no-headers | head -n 3	Alternativa sin awk
3.3	ps -eo pid,ppid,stat,comm	Estados de los procesos
3.3	man ps | sed -n '/^PROCESS STATE CODES/,/^OBSOLETE SORT KEYS/p'	Códigos de estado
3.4	pgrep -n sha256sum ; pstree -p -s $(pgrep -n sha256sum)	Árbol del último sha256sum
3.4	sleep 300 & ; pstree -p -s -l $!	Relación padre-hijo

Fase	Comando	Para qué
		directa
3.5	pgrep sha256sum | xargs kill -s SIGTERM	Limpiar la carga
4.1	nano servicio_senales.sh	Crear el script
4.2	ls -l servicio_senales.sh ; chmod u+x servicio_senales.sh	Reto de permisos
4.3	kill -2 1318 ; kill -15 1318	Probar SIGINT y SIGTERM
4.4	kill -l ; man 7 signal ; kill -9 1318	Identificar y usar SIGKILL
5.1	sleep 10000 > /dev/null 2>&1 & ; echo $!	Proceso con flujos redirigidos
5.2	cd /proc/$P/ ; ls -l fd/	Descriptores de archivo
5.3	grep -i "switches" status	Cambios de contexto
5.4	ps -p $P -o pid,ppid,stat,comm ; grep -E "^(Name|State|Pid|PPid):" status	ps frente a /proc
5.4	strace -e trace=openat,open,openat2 ps -p $P -o pid,ppid,stat,comm	Ver qué lee ps
5.5	cat limits | head -n 12	Límites de recursos
5.6	ls -lh status ; cp status ~/status_copia.txt ; wc -c ...	Pseudo-archivo frente a copia
5.7	kill -SIGKILL $P ; ps -p $P ; ls /proc/$P	Destruir y verificar
Extra	bash -c 'sleep 1 & exec sleep 60' &	Reproducir un zombie

Anexo B. Script servicio_senales.sh

```
#!/bin/bash

# Captura de SIGINT y SIGTERM
trap 'echo "[!] Petición de terminación interceptada e ignorada."' SIGINT SIGTERM

echo "Servicio de pruebas iniciado. PID de este proceso: $$"

while true; do
    sleep 2
done
```